Coursework Specification

COMP1858

Data Structures and Algorithms

2025-2026

Contents

COMP1858 Coursework Overview 3
Coursework Submission Requirements 4
Coursework Specification 5
Detailed Specification and Deliverables 7
Report Presentation 12
Assessment Criteria 13

COMP1858 Coursework Overview

Module Leader: Dr Ebrahim Patel.

Deadline Date: 30/03/2026, 17:00 UK time.

Contribution: 100% of module.

Pass mark: 40%.

Marking: Feedback and grades are normally made available within 15 working days (usually 21 calendar days, unless there are bank holidays) of the coursework deadline. Your coursework may or may not be marked anonymously, depending on requirements.

Learning Outcomes

1. Demonstrate detailed understanding of various data structures and the corresponding algorithms, which manipulate them.
2. Design, develop and test software solutions, which successfully process a wide range of data structures by employing various algorithms.
3. Further develop well-established algorithms into advanced software solutions, which solve real-world data-related problems.

Plagiarism

Plagiarism is presenting somebody else's work as your own. It includes copying information directly from the Web or books without referencing the material; submitting joint coursework as an individual effort; copying another student's coursework; stealing coursework from another student and submitting it as your own work. Suspected plagiarism will be investigated and if found to have occurred will be dealt with according to the procedures set down by the University. Please see your student handbook for further details of what is / isn't plagiarism.

All material copied or amended from any source (e.g., internet, books, AI) must be referenced correctly according to the reference style you are using.
Your work will be submitted for plagiarism checking. Any attempt to bypass our plagiarism detection systems will be treated as a severe Assessment Offence.

Generative AI tools

We recommend giving lots of thought before using genAI. Using genAI is not strictly prohibited if it is for brainstorming or getting past the ‘blank page’ block, but we recommend always critically reflecting on the output, since it is not guaranteed to be factual or correct. If you do use generative AI, you need to reference it and put any text **and code** within quotation marks (otherwise it’s an academic offence). You must also declare your use of genAI by appending the AI Declaration of Use to your work (see next section).

Coursework Submission Requirements

An electronic copy of your work for this coursework must be fully uploaded on the **Deadline Date of 30th March 2026** using the link on the coursework Moodle page for COMP1858. For this coursework you must submit 2 separate files, **combined into one .zip file**:

- **A single pdf file named 'STUDENTID_report_comp1858.pdf'** which will be the written report. The length of this written report should be **approximately 3500 words (or around 7-8 A4 pages in Arial font, size 12)**. The length excludes references and appendices, if you have any. **Do not include your name** in the report.
- **A single file** containing your Python work (ideally in .py script format). The file should be named 'STUDENTID_code_comp1858.py'. If submitting a .ipynb file, please save it after executing **all** the cells, so that it shows the output. Your code should show all and only all the relevant implementation steps needed to reproduce the content of your report.

Please note:

- Any text in the document must not be an image (i.e. must not be scanned).
- Make sure that any files you upload are virus-free and not protected by password or corrupted, otherwise they will be treated as null submissions.
- You must NOT submit a paper copy of this coursework.
- Coursework can only be accepted if it is submitted via the appropriate Moodle link.

The University website has details of the current Coursework Regulations, including details of penalties for late submission, procedures for Extenuating Circumstances, and penalties for Assessment Offences. See http://www2.gre.ac.uk/current-students/regs

Declaration of AI Use

Please **copy and paste the text below** to the beginning of your assignment (select YES or NO where relevant). Failure to complete this task may result in a penalty.

I have used <insert AI tool names> while undertaking my assignment in the following ways:

- To develop research questions on the topic: YES/NO
- To create an outline of the topic: YES/NO
- To explain concepts: YES/NO
- To support my use of language: YES/NO
- To summarise the following articles/resources: <add relevant articles/resources>
- In other ways, as described below: <list the other ways, e.g., with coding>

Coursework Specification

Task 1: Recipes and Ingredients

The Set Menu is a recipe app that helps users quickly find recipes from around the world based on ingredients that they supply. You are consulting for The Set Menu to improve the algorithms that power the app. The company has supplied you with two datasets – Dataset 1, containing ingredients used in common recipes around the world and Dataset 2, containing pairs of ingredients that share 'flavour compounds', i.e., that are likely to taste well together. There are over 50,000 recipes in the first dataset, and over 220,000 pairs in the second.

The Set Menu would like you to write the results of your analysis, including use of Python to implement the algorithms, in a report. Specific details about the tasks and what to include in the report are provided in the next section.

The datasets can be downloaded from this Moodle link, and has been adapted to the requirements of this module (the original dataset is freely available under the CC0: Public Domain licence).

Dataset 1: Recipe ingredients around the world

Each row in the dataset corresponds to one recipe/dish.

Column 1	Region of the world (e.g., Middle Eastern)
Column 2	Ingredient
Column 3	Ingredient
Column 4	Ingredient
…	…

E.g., for row 13 (= dish 13), there are 12 ingredients, stored in 12 columns.

Dataset 2: Flavour compounds

Each row in the dataset corresponds to one recipe/dish.

Column 1	Ingredient 1
Column 2	Ingredient 2
Column 3	Number of common flavour compounds ('flavour compound score')

E.g., coffee (Ingredient 1) and grilled_beef (Ingredient 2) share 106 flavour compounds, whilst apple and cheddar_cheese share 79 flavour compounds.

Note: The algorithms that you construct for The Set Menu are very important. If you make a mistake, it could cost many thousands of pounds, so you should think of this role as a big responsibility. Overall, you must be able to justify any decision that you make – it is unacceptable to say, for instance, "ChatGPT gave me this reason" without convincingly providing the background (ChatGPT can make many mistakes).

Task 2: Planning a World Tour. (Travelling Salesperson Problem)

The Algos are a popular British rock band who are planning a world tour in 2027. You are on their management team and are tasked with planning the most efficient tour in terms of cost and distance. The tour will start and end in the band's home country of the UK. More specifically, the band will start in the UK, playing in 3 cities. They will then tour the world, playing in 3 cities in the continents below, before returning to the UK to finish in the UK city where they started. In total, The Algos will play in 15 cities on this world tour. Here is the list of cities to be considered in each continent:

UK: (0) Belfast, (1) Birmingham, (2) Bristol, (3) Cardiff, (4) Edinburgh, (5) Leeds, (6) Liverpool, (7) London, (8) Manchester, (9) Newcastle.

North/South America: (0) Boston, (1) Buenos Aires, (2) Dallas, (3) Los Angeles, (4) Mexico City, (5) New York, (6), Rio de Janeiro, (7) Santiago, Chile (8) Toronto, (9) Vancouver.

Europe (not including the UK): (0) Athens, (1) Barcelona, (2) Dublin, (3) Istanbul, (4) Lisbon, (5) Munich, (6) Paris, (7) Rome, (8) Stockholm, (9) Vienna.

Africa: (0) Accra, (1) Alexandria, (2) Cairo, (3) Cape Town, (4) Casablanca, (5) Johannesburg, (6) Kinshasa, DR Congo (7) Lagos, (8) Nairobi, (9) Tunis.

Asia: (0) Auckland, (1) Beijing, (2) Delhi, (3) Kuala Lumpur, (4) Lahore, (5) Melbourne, (6) Mumbai, (7) Seoul, (8) Singapore, (9) Tokyo.

Note: The World Tour - and the algorithm that you construct to create it – is obviously very important. If you make a mistake, it could cost hundreds of thousands of pounds, so you should think of this role as a big responsibility. Overall, you must be able to justify any decision that you make in planning this tour – it is unacceptable to say, for instance, "ChatGPT gave me this reason" without convincingly providing the background (ChatGPT can make many mistakes).

Detailed Specification and Deliverables

The specific tasks for this assignment are given in the sections below. These sections also make up the report that you should submit, which details your work (see also Coursework Template). Lengths of sections (in words) are indicated as a guide only – the total length should adhere to the Coursework Submission Requirements.

Task 1: Recipes and Ingredients

1.1. Data Assessment

[200 words] Describe the data types in the two datasets, ideally in the form of a table.

You should reference at least one lecture week (topic) and slide number.

1.2. Data Structures

[300 words] For each dataset, **choose one** of the following three data structures to store and manipulate the data, and clearly show - using small examples - how you can represent the dataset as your chosen data structure.

- Adjacency List.
- Adjacency Matrix.
- Dictionary.

You should reference at least one lecture week (topic) and slide number.

1.3. Recipe Algorithm

[500 words] Write an algorithm - call it **Algorithm 1.3** - that takes in any number of ingredients as input and uses **Dataset 1** to return all recipes/dishes that contain those ingredients. The returned dishes should be sorted so that the dishes that contain the highest proportion/percentage of the input ingredients appear at the top.

<u>Example:</u> if my input ingredients are: olive_oil, potato, beef, and garlic, then the dish that appears at the top (the 'most recommended dish') might be a dish that contains only these four ingredients (100% match), followed by, perhaps, a dish that contains olive_oil, onion, potato, beef, and garlic (80% match).

Write your algorithm in pseudo-code and/or describe your algorithm's main steps in non-technical language suitable for a reader who is unfamiliar with Python. You should use only the algorithms seen in class and the provided Python scripts to develop your algorithm and code. Clearly show how you built your algorithm and describe your thought process. Screenshots of code longer than 1 line should not be included.

You should reference at least one lecture week (topic) and slide number.

1.4. Improved Recipe Algorithm

[500 words] Now use **Dataset 2** (flavourcompounds.csv) to **develop an algorithm that improves on Algorithm 1.3** – call this new algorithm **Algorithm 1.4**. You are strongly encouraged to consider what the user might want to see, and remember the world regions. One suggestion is: Given a set of input ingredients, Algorithm 1.4 recommends recipes whose ingredients have the highest 'flavour compound scores' between them; these recipes may be regarded as the ones with the best flavour.

<u>Example:</u> if my input ingredients are: olive_oil, potato, beef, and garlic, then Algorithm 1.4 runs Algorithm 1.3, takes its output, and then:

1. For each dish, finds all pairs of ingredients, e.g., if the dish is {olive_oil, onion, potato, beef, garlic}, the pairs of ingredients are: (olive_oil, onion), (olive_oil, potato), (olive_oil, beef), (olive_oil, garlic), (onion, potato), (onion, beef), (onion, garlic), (potato, beef), (potato, garlic), (beef, garlic).
2. Adds all the 'flavour compound scores' together.
3. Returns: the dish with the largest total flavour compound scores.

In this task, you should first consider simplifying the ingredients in flavourcompounds.csv, so that similar ingredients are regarded as the same. E.g., camembert_cheese, cheddar_cheese, and swiss_cheese may all be simplified to 'cheese'. Following this step, you should create a new list of flavour compounds with the simplified ingredients and new numbers (taken as totals of the previous corresponding numbers).

Write your algorithm in pseudo-code and/or describe your algorithm's main steps in non-technical language suitable for a reader who is unfamiliar with Python. You should use only the algorithms seen in class, the provided Python scripts, and Algorithm 1.3 to develop your improved algorithm and code. Clearly show how you built your algorithm using Algorithm 1.3, and describe your thought process. Screenshots of code longer than 1 line should not be included.

You should reference at least one lecture week (topic) and slide number.

1.5. Examples

[300 words] Demonstrate the effectiveness of Algorithm 1.4 by choosing any 5 ingredients and showing clearly the input, output and brief description of the steps. Show also a counter-

example, i.e., an example where the algorithm does ***not*** work or fails to produce the desired outcome, and demonstrate why.

Task 2: Planning a World Tour

2.1. Data Selection and Structures

[300 words] Clearly **state your student ID**. Take the last three digits of your ID and, for each continent, select those cities that correspond to these three digits. E.g., if the last three digits of your ID are 398, then you should select the 3rd, 9th, and 8th cities as follows:
UK: (3) Cardiff, (8) Manchester, (9) Newcastle.
North/South America: (3) Los Angeles, (8) Toronto (9) Vancouver.
Europe (not including the UK): (3) Istanbul, (8) Stockholm, (9) Vienna.
Africa: (3) Cape Town, (8) Nairobi, (9) Tunis.
Asia: (3) Kuala Lumpur, (8) Singapore, (9) Tokyo.
Your world tour will then comprise the above 15 cities.

Find the distances and costs between all cities in your world tour (you should consult appropriate sources and cite all references). **Describe all data types and choose one suitable data structure** (from those that you have seen in class) to store this data, justifying your choice.

You should reference at least one lecture week (topic) and slide number.

2.2. The World Tour Algorithm

[600 words] Construct an algorithm – call it Algorithm 2.2 - that produces a world tour for The Algos. The objective is to minimise the total distance travelled and/or the cost. Clearly state any assumptions that you make (e.g., the band can travel on any day, distances are accurate to 10 km, etc.)

For this task, aside from Python's in-build functions, you should use only the provided Python scripts to develop your algorithm and code. Clearly show how you built your algorithm using the provided code and your choice of data structure, and describe your thought process. Write your algorithm in pseudo-code and/or describe your algorithm's main steps in non-technical language suitable for a reader who is unfamiliar with Python. Screenshots of code longer than 1 line should not be included.

One approach is to consider the **'Nearest Neighbour Algorithm'** – you should look this up in an academic resource (and cite it appropriately); the algorithm is also given as the Python

function **`nearest_neighbour_tsp(dist, start)`**, where **`dist`** is a two-dimensional list of distances and **`start`** is the starting city. Note, however, that, while this function does work, it ***produces an incorrect total distance – you should check this and correct it before using this code.***

Similarly, another approach is to extend **'Dijkstra's Algorithm'**, which finds the shortest path between any two nodes (cities) in a graph; the algorithm is also given as the Python function **`dijkstra(graph, start_node)`**, where **`graph`** is a dictionary that encodes all direct distances between nodes and **`start_node`** is the starting city. Note, however, that, while this function does work, it ***produces incorrect shortest distances – you should check this and correct it before using this code.***

N.B., this is a famous computationally hard problem that is unsolved. This means you will not find a ready-made solution (using the tools of this course) anywhere. The best you can do is to develop algorithms that are 'good enough'.

You should reference at least one lecture week (topic) and slide number.

2.3 Amending the World Tour

[300 words] Suppose there has been a late change because of transport issues in Europe. A European city is to be changed to a European city with the next index (mod 10). E.g., if your last student ID digit is 4 (Lisbon), then the new city has index 5 (Munich); if your last ID digit is 9 (Vienna), then the new city has index 0 (Athens).

Demonstrate how your algorithm allows for this possibility of changes. You should first show exactly which parts of the algorithm are affected, then demonstrate the outcome of this change in cities, i.e., show the resulting world tour, along with total distance or cost.

You should reference at least one lecture week (topic) and slide number.

---

3. Time Complexity

[300 words] Choose **one** of your **three algorithms** - Algorithm 1.3, 1.4, or 2.2 - and show that its time complexity is $O(n)$, $O(n^2)$, $O(\log n)$, or $O(n \log n)$, where n is the size of the dataset that you consider. If you choose Algorithm 1.3 or 1.4, select any five subsets of the corresponding dataset(s), sizes $n = 10, 20, 100, 500$ and 1000 (e.g., 10 dishes, 20 dishes, 100 dishes, 500 dishes, and 1000 dishes). If you choose Algorithm 2.2, select any five subsets of the 50 cities, sizes $n = 5, 15, 25, 35,$ and 50 (i.e., 5 of the given cities, 15 cities, 25 cities, 35 cities, and all 50 cities); By applying your algorithm to each of these subsets, demonstrate

the algorithm's time complexity; produce a table and/or graph of your results. Fully show your working and justify your selection of time complexity in non-technical language.

You should reference at least one lecture week (topic) and slide number.

---

4. Conclusions

[200 words] Choose **one** of **Algorithm 1.4** or **Algorithm 2.2** and summarise it for a non-expert reader. You should briefly describe the data structure(s), what the algorithm does, and how well the algorithm performed. State also how you might improve your algorithm – what further information might you need or what would you do differently if you had more time?

---

5. References

This section is to list the references and sources that you used. These can be academic papers, blogs, code repositories, and more. Remember to give credit if someone else's work has helped you complete your coursework. You must use Harvard referencing.

6. Appendices

If needed, this section should contain such things as **portions of code** or **large tables of data** that are not crucial for the assessment or the report but are valuable to give the interested reader more detail. E.g., you may include a table of Middle-Eastern recipes under 'Appendix A'.

Report Presentation

Please note the following for the presentation of your work.

a) Your report can be written in MS Word, but you must remember to **save it as a pdf before submitting**. You are advised to use the provided template, which contains all the necessary sections.

b) You should write for someone with the knowledge one would expect of a **first-year computing undergraduate**. You might find it helpful to imagine writing as if you are teaching this course to a first-year student.

c) The report should be written in a professional style, in English, with clear language and a logical flow. Double-check your spelling, punctuation, and grammar. You might get useful feedback from the Studiosity service, available through the course Moodle page.

d) In principle, the reader should be able to carry out the steps of your algorithms based on the report alone, without needing to look at the code. To achieve this, all the necessary implementation details must be included in the report (in Appendices, if appropriate).

e) The report must include all of the Sections 1-5 and 6 (Appendices) if necessary, but you are allowed to add sub-sections (with appropriate subheadings). References and Appendices will not be included in the report length.

f) All figures must be captioned with legible text and an appropriate label – use the same label to reference the figure in the main report text, e.g., "Fig. 3 shows something interesting" should be written in the main text, while the figure's caption might read "Figure 3: Something interesting". Small screenshots of code, such as 1-line output, are acceptable; larger screenshots should only be included in the appendices.

Code

The code must be developed in Python, mainly using the provided code in the section entitled, 'Submission Guidance and Grading Criteria for Assessments' on Moodle. Your code should clearly show how you used or changed the provided code, and it should show the implementation leading to all the results described in the coursework. If needed, tutors should be able to run the code without errors. Tutors may review your code to check that the implementation is consistent with what is written in the report. Inconsistencies between the report and the code will be penalised. However, **the code itself is not marked**. Only what is described in your report counts towards your final mark.

Assessment Criteria

Marks will typically require the following.

80 to 100: An outstanding and fully practical solution, showing close to complete working algorithms. The data structures suit the algorithms and time complexity backs up the final proposal. Many ideas are innovative, and clearly build on the provided Python code. Most of the report can be followed and worked through by a first-year student. Lecture slides are correctly cited throughout.

70 to 79: A good and practical solution, showing close to complete working algorithms. The data structures suit some of the algorithms and time complexity backs up the final proposal. Some ideas are innovative and build on the provided Python code. Around half the report can be followed and worked through by a first-year student. Lecture slides are correctly cited throughout.

60 to 69: A satisfactory but practical solution, showing working algorithms that may be incomplete and confusing in some places. The data structures suit some of the algorithms but it may be unclear how the time complexity backs up the final proposal. Some ideas are innovative but not fully clear on how they build on the provided Python code. Up to half the report can be followed and worked through by a first-year student. Lecture slides are correctly cited in most places.

50 to 59: A satisfactory but somewhat impractical solution, showing working algorithms that may be incomplete and confusing in some places. The data structures suit some of the algorithms but it is unclear how the time complexity backs up the final proposal. There are no or few innovative ideas, and it is unclear how they build on the provided Python code. A first-year student would find difficulty in following and working through some important areas of the report. Lecture slides are correctly cited in few places.

40 to 49: A minimal and somewhat impractical solution, showing algorithms that may be incomplete and confusing in some places. The data structures suit only a few of the algorithms and it is unclear how the time complexity backs up the final proposal. There are no or very few innovative ideas, and it is largely unclear how they build on the provided Python code. A first-year student would find difficulty in following and working through some important areas of the report. Lecture slides are not correctly cited in most places.

0 to 39: An impractical solution, showing algorithms that are largely incomplete and confusing in many places. The data structures may not be addressed or suit only a few of the algorithms, whilst the time complexity may not be carried out. There are no innovative ideas, and the provided Python code is largely disused. Lecture slides are not or incorrectly cited in most places. As a whole, a first-year student would fail to follow and work through the report.

Marks breakdown

[10 marks] 1.1 and 1.2. Data Assessment and Data Structures **Marks**

Identification and justification of data structures and types. [0-4] No or little identification of data structures and types, nor justification. [5-8] Sufficient identification, but little justification. [9-10] Excellent identification of data structures and types; sufficient justification.	10

[30 marks] 1.3, 1.4, 1.5. Algorithms 1.3 and 1.4

Real-world plausibility of both algorithms. [0-1] No, unclear or insufficient descriptions of algorithms. [2-3] Clear descriptions of algorithms, real-world motivation unclear under some scenarios. [4-5] Excellent, plausible, algorithms that make real-world sense.	5
Algorithm practicality, i.e., can the algorithmic solutions be worked through by a (non-expert) first-year student? [0-2] Algorithms are largely absent or incorrect, and practicality unclear. [3-5] Some correct parts in algorithms, practicality addressed but largely unclear. [6-7] Most parts of algorithms correct and practical; some errors that may be confusing. [8-10] All or most parts of algorithms correct and practical; few minor errors.	10
Application of algorithms in examples. [0-4] No or insufficient examples of algorithm application. [5-8] Sufficient examples of algorithm application, but may be intangible or contain errors. [9-10] Sufficient and tangible examples of algorithm application; few minor errors.	10
Novelty of proposed algorithms. [0-1] Unoriginal proposal. [2-3] Original/novel proposal, but execution unclear. [4-5] Original/novel proposal and execution.	5

[10 marks] 2.1. Data Selection and Structures

Correct cities chosen based on student ID.	1
Identification and justification of data structures and types. [0-3] No or little identification of data structures and types, nor justification. [4-7] Sufficient identification of structures, but little justification. [8-9] Excellent identification of data structures and types; sufficient justification.	9

[25 marks] 2.2 and 2.3. Algorithm 2.2 and its Flexibility

Real-world plausibility of algorithm. [0-1] No, unclear or insufficient descriptions of algorithm. [2-3] Clear descriptions of algorithm, real-world motivation unclear under some scenarios. [4-5] Excellent, plausible, algorithm that makes real-world sense.	5
Algorithm flexibility and practicality, i.e., can the algorithmic solutions be worked through by a (non-expert) first-year student? 0-2] Algorithm is largely absent or incorrect, and practicality unclear. [3-5] Some correct parts in algorithm, practicality addressed but largely unclear. [6-7] Most parts of algorithm correct and practical; some errors that may be confusing. [8-10] All or most parts of algorithm correct, flexible, and practical; few minor errors.	10
Application of algorithm in examples. [0-1] No or insufficient examples of algorithm application. [2-3] Sufficient examples of algorithm application, but may be intangible or contain errors. [4-5] Sufficient and tangible examples of algorithm application; few minor errors.	5
Correct use of provided Python code. [0-1] Provided Python code is not corrected, and used little to develop algorithm. [2-3] Provided Python code is not fully corrected, but used to develop algorithm. [4-5] Provided Python code is corrected, then used to develop algorithm.	5

[10 marks] 3. Time Complexity

Time complexity analysis. [0-4] No or little analysis of algorithmic time complexity. [5-8] Sufficient analysis of time complexity, perhaps including graphs and examples, but with errors/omissions. [9-10] Excellent analysis of time complexity, including graphs and examples; few minor errors.	10

[5 marks] 4. Conclusions

Summary of work done and identification of improvements. [0-1] No or little summary of data structures and/or algorithm. No suggested improvements. [2-3] Good summary of data structures and/or algorithm. Suggested improvements are minimal. [4-5] Excellent summary of data structures and algorithm. Suggested improvements are relevant and realistic.	5

[10 marks] Writing Clarity and Report Presentation

[0-4] Unsatisfactory clarity and structure. Report is often hard to follow. [5-8] Sufficiently clear, correct structure and referencing, most lecture citations correct. Some sections may be hard to follow for a non-expert. [9-10] Largely clear for a non-expert reader, good structure and referencing, lectures correctly cited.	10